

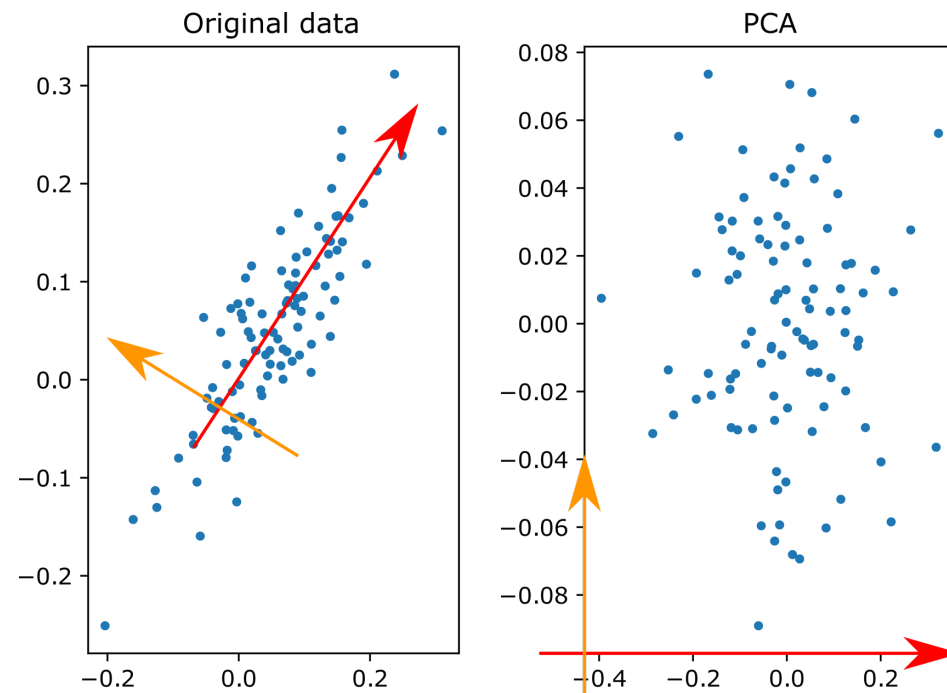
Dimensionality Reduction

Dimensionality Reduction

- In many experiments the number of variables measured is overwhelming... are all these variables relevant ?
- Can we project the data to a lower dimensional space (losing as little information as possible) ?
- Why doing dimensionality reduction
 - many algorithms are inefficient when the number of dimensions is high (curse of dimensionality)
 - for visualization
- Example in biomedical engineering
 - EEG systems record 32–256 channels with high temporal resolution
 - Gene expression datasets (e.g., RNA-seq or microarrays) can contain 10,000–60,000 genes per sample
 - Wearable sensors collect large streams of physiological data
 - ...

Principal Component Analysis (PCA)

- Principal Component Analysis: the original set of variables are transformed into a set of linearly uncorrelated variables
- It makes a linear projections of the features into the directions of maximum variance



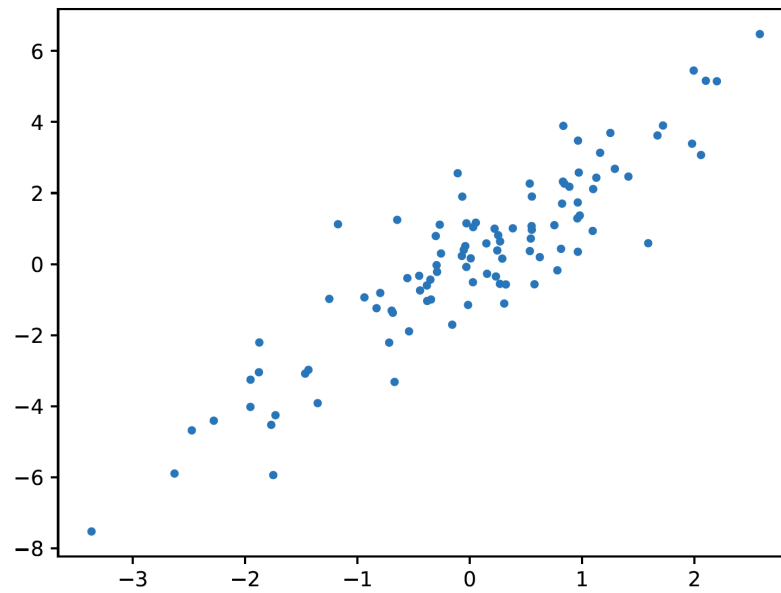
number of features
 n x n
 covariance matrix

1st feature
 average of the 1st feature


$$COV = \begin{pmatrix} E[(x_0 - \mu_0)(x_0 - \mu_0)] & \dots & E[(x_0 - \mu_0)(x_{n-1} - \mu_{n-1})] \\ \dots & \dots & \dots \\ E[(x_{n-1} - \mu_{n-1})(x_0 - \mu_0)] & \dots & E[(x_{n-1} - \mu_{n-1})(x_{n-1} - \mu_{n-1})] \end{pmatrix}$$

correlation matrix

$$CRR = \begin{pmatrix} \frac{COV_{0,0}}{\sqrt{COV_{0,0}COV_{0,0}}} & \dots & \frac{COV_{0,n-1}}{\sqrt{COV_{0,0}COV_{n-1,n-1}}} \\ \dots & \dots & \dots \\ \frac{COV_{n-1,0}}{\sqrt{COV_{n-1,n-1}COV_{0,0}}} & \dots & \frac{COV_{n-1,n-1}}{\sqrt{COV_{n-1,n-1}COV_{n-1,n-1}}} \end{pmatrix}$$



$$COV = \begin{matrix} 0.98 & 2.02 \\ 2.02 & 5.33 \end{matrix}$$

$$CRR = \begin{matrix} 1.0 & 0.88 \\ 0.88 & 1.0 \end{matrix}$$

$$X = \begin{pmatrix} x_0^0 & \dots & x_0^{m-1} \\ \vdots & \ddots & \vdots \\ x_{n-1}^0 & \dots & x_{n-1}^{m-1} \end{pmatrix}$$

number of samples
 $\mu_i = 0$
 average value of dimension i
 1st sample (x^0)
 number of dimensions

$$COV = \frac{1}{m}XX^T$$

$$X' = R^T X$$

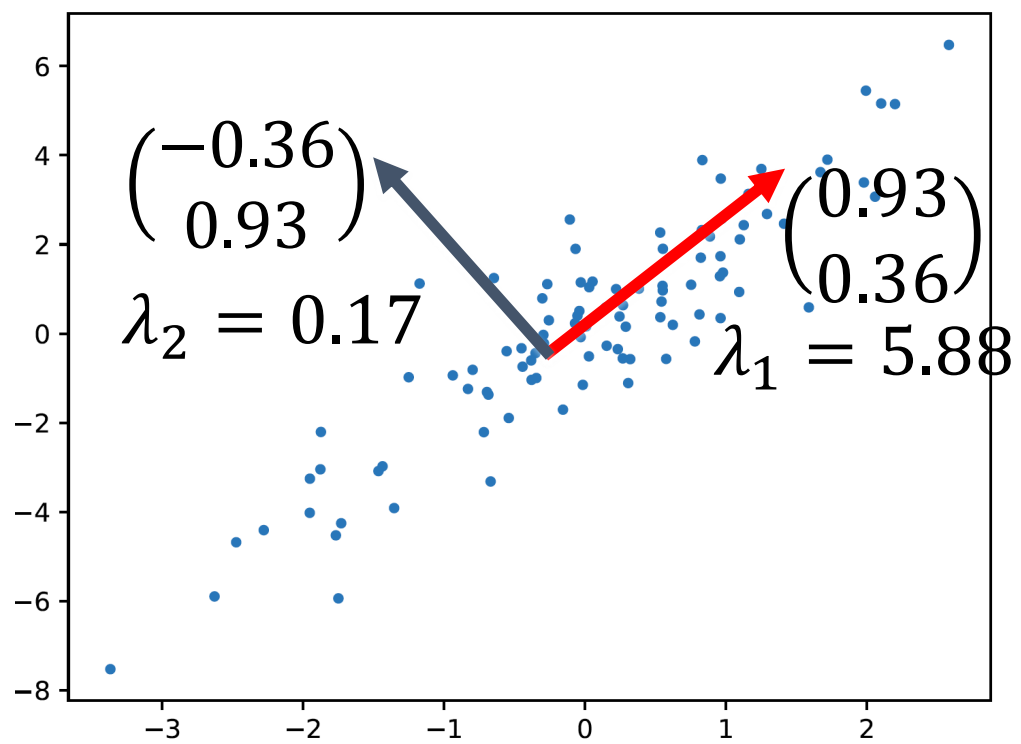
Samples projected in the space of the eigenvectors

eigenvalues (in descending order)

$$R = \begin{pmatrix} \lambda_1 & \dots & \lambda_n \\ | & \dots & | \\ e_1 & \dots & e_n \\ | & \dots & | \end{pmatrix}$$

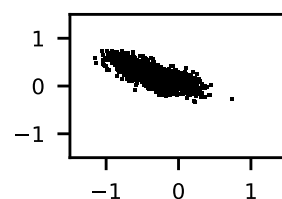
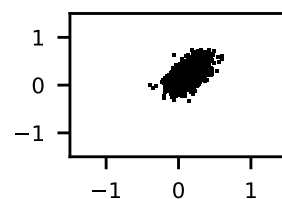
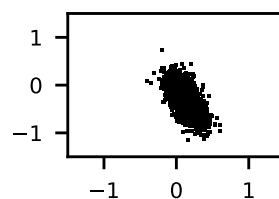
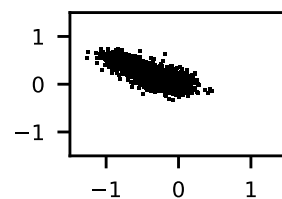
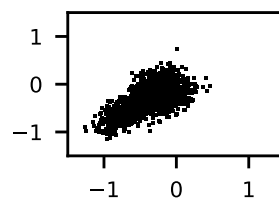
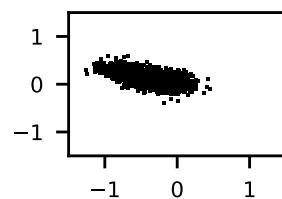
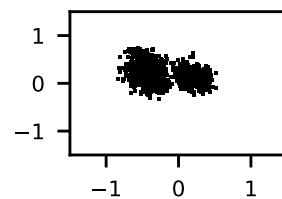
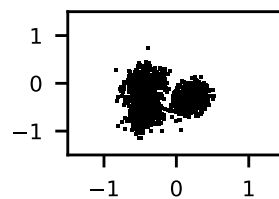
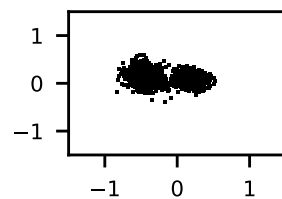
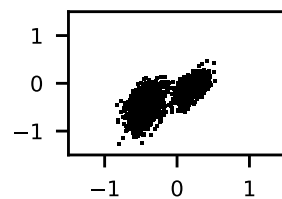
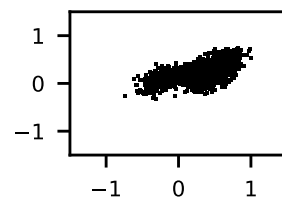
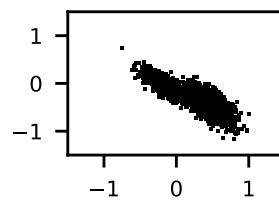
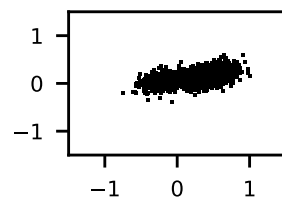
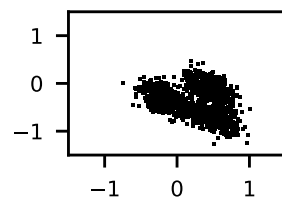
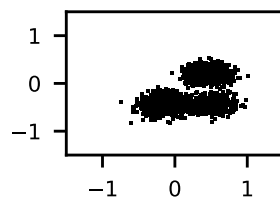
eigenvectors of the covariance matrix

$$\begin{aligned}
 COV' &= \frac{1}{m}X'X'^T = \frac{1}{m}R^T X(R^T X)^T = \frac{1}{m}R^T XX^T R = \frac{1}{m}R^T (mCOV)R \\
 &= R^T \begin{pmatrix} | & \dots & | \\ \lambda_1 e_1 & \dots & \lambda_n e_n \\ | & \dots & | \end{pmatrix} = \begin{pmatrix} \lambda_1 & 0 & \dots & 0 \\ 0 & & \ddots & \\ \vdots & & & 0 \\ 0 & \dots & 0 & \lambda_n \end{pmatrix}
 \end{aligned}$$

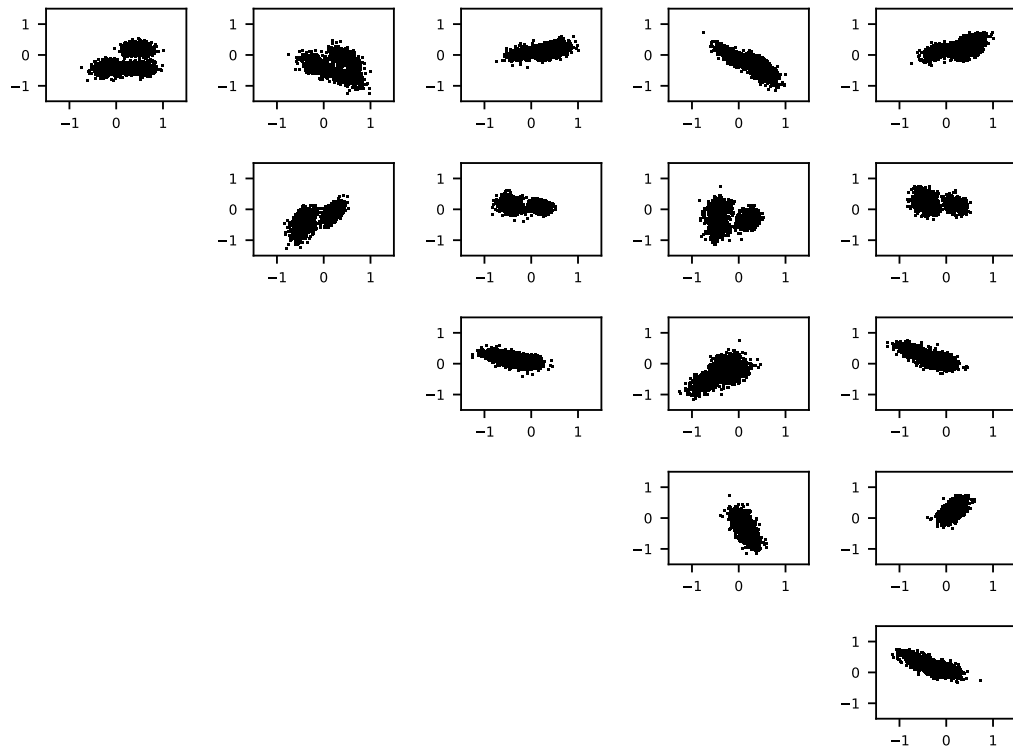


Eigenvectors / Values of the covariance matrix

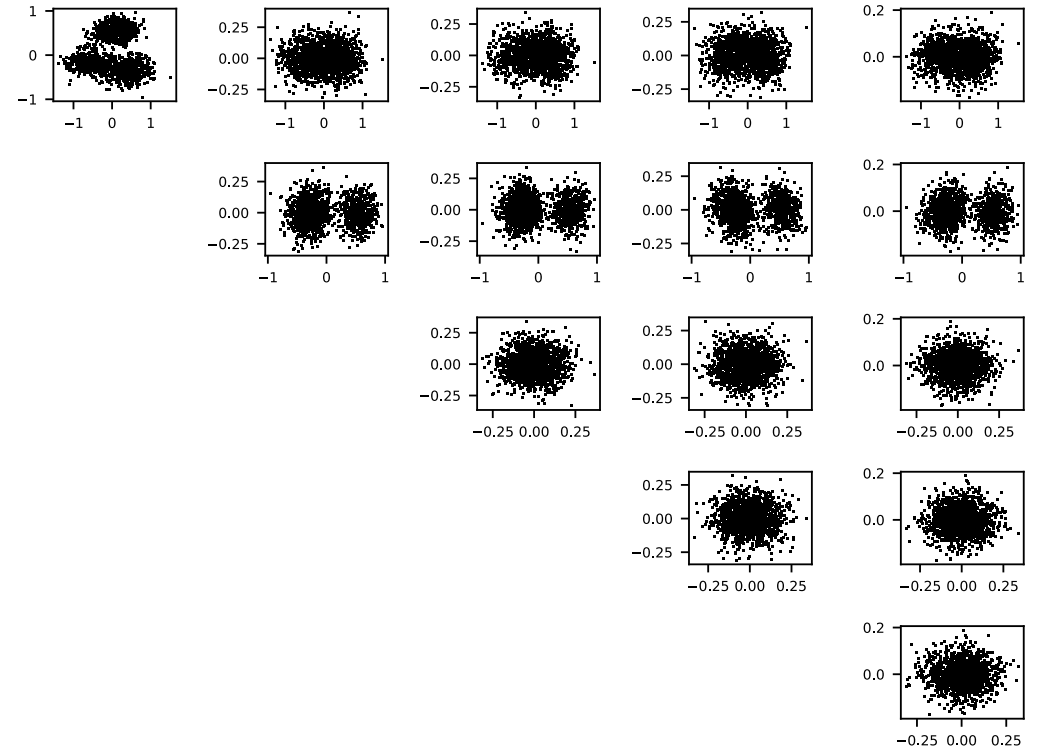
- The first eigenvalue (i.e. the one associated with the highest eigenvalue) is the direction of maximum variance
- The second eigenvalue is the second direction of maximum variance orthogonal to the first eigenvalue, and so on...

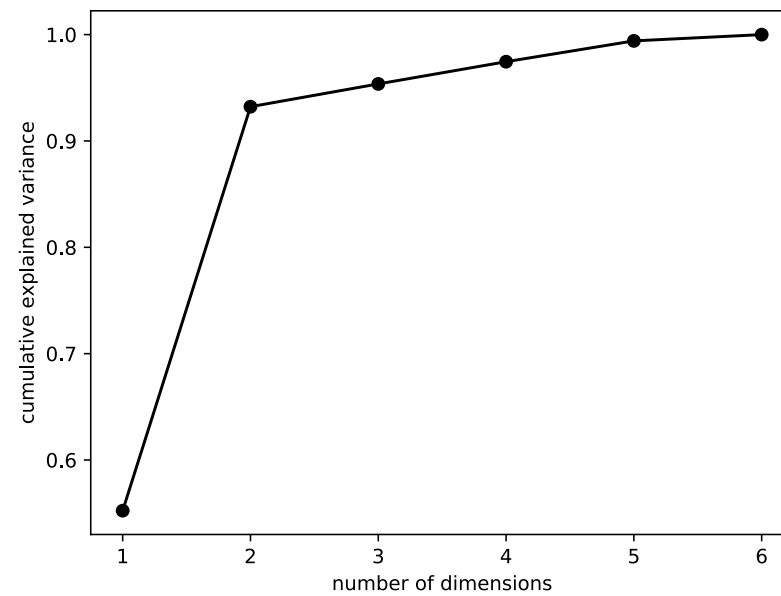
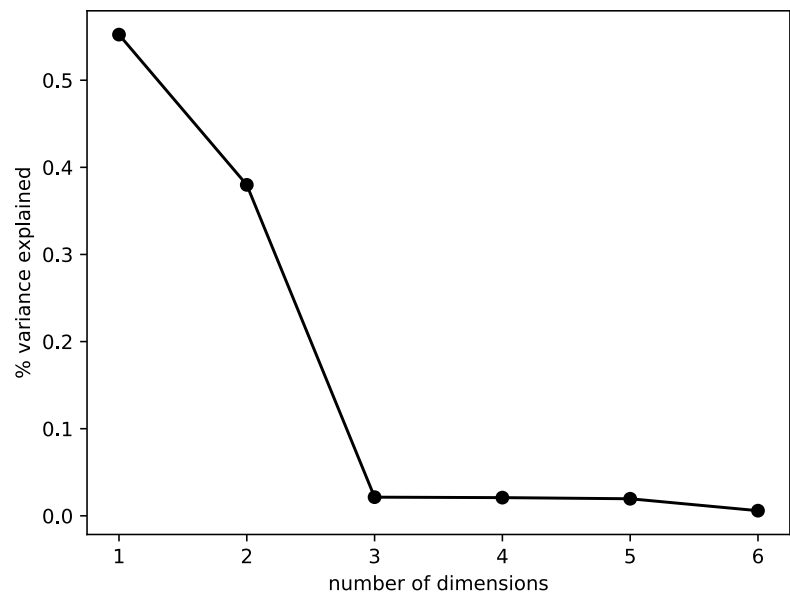
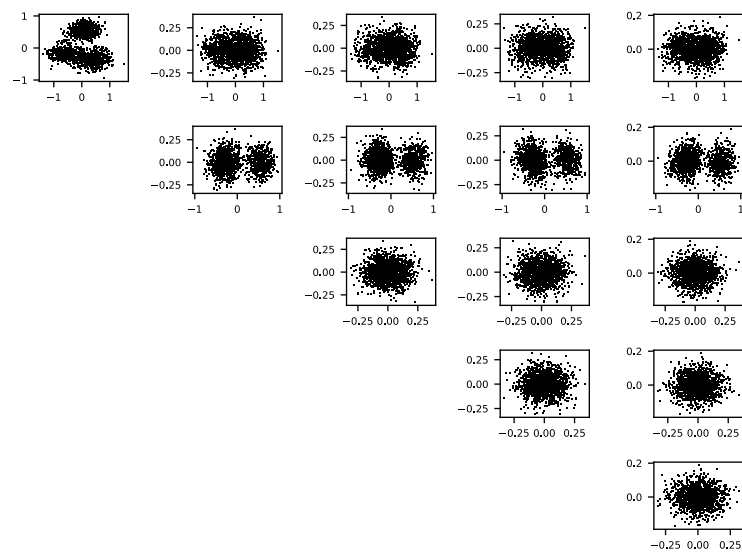


Original data



PCA

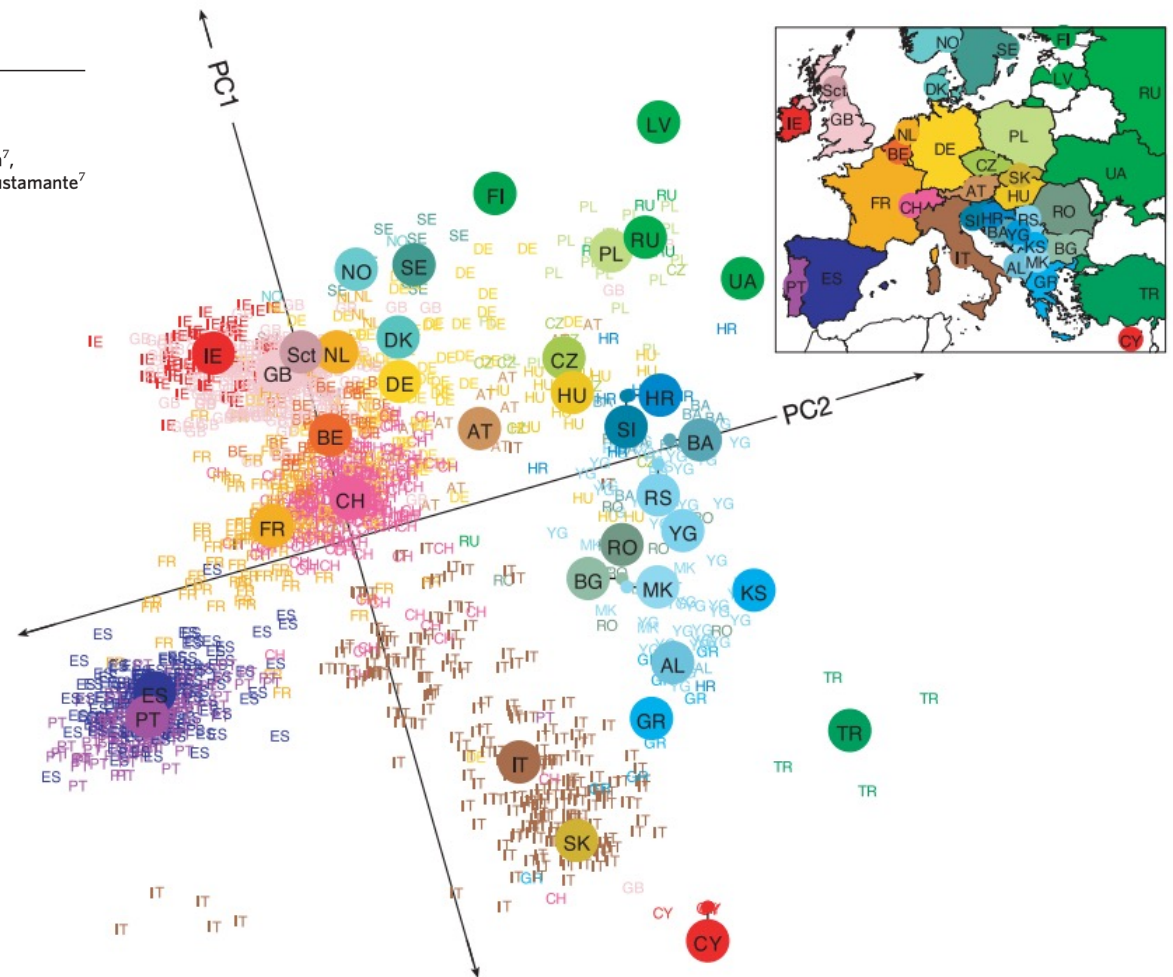




LETTERS

Genes mirror geography within Europe

John Novembre^{1,2}, Toby Johnson^{4,5,6}, Katarzyna Bryc⁷, Zoltán Kutalik^{4,6}, Adam R. Boyko⁷, Adam Auton⁷, Amit Indap⁷, Karen S. King⁸, Sven Bergmann^{4,6}, Matthew R. Nelson⁸, Matthew Stephens^{2,3} & Carlos D. Bustamante⁷



PCA pros and cons

- Efficient
- Easily interpretable
- It's possible to project new data on previously define PC
- Linear method → it cannot reduce dimensions if the relations among data are not linear

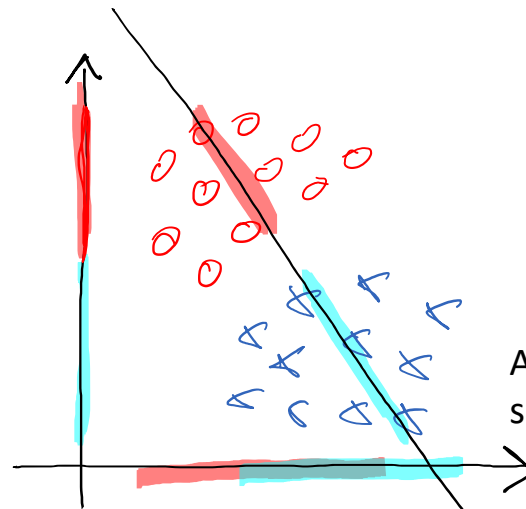
Linear Discriminant Analysis

Linear Discriminant Analysis (LDA) is a **supervised** dimensionality reduction technique that projects data onto a lower-dimensional space while:

- maximizing the distance between class means
- minimize the variance within each class

Unlike PCA, which focuses on maximizing variance, LDA focuses on maximizing class separability.

Along this direction the two distributions are separated but the variances are high → bad projection



Along this direction the two distributions superimpose → bad projection

Along this direction the averages are separated and the variances are low → it's a good projection for separating the classes

$$X = \begin{pmatrix} x_0^0 & \cdots & x_0^{m-1} \\ \vdots & \ddots & \vdots \\ x_{n-1}^0 & \cdots & x_{n-1}^{m-1} \end{pmatrix}$$

number of samples
 number of dimensions
 $x^j = 0$ j-th sample

$$y = (y^0 \quad \cdots \quad y^{m-1})$$

y^j the class of the j-th sample

$$\mu_k = \frac{1}{m_k} \sum_{x \in \mathcal{C}_k} x$$

average of the samples in class k
 number of the samples in class k
 samples that belong to class k

$$\mu = \frac{1}{m} \sum_{k=0}^K m_k \mu_k$$

$$S_w = \sum_{k=0}^K \sum_{x \in C_k} (x - \mu_k)(x - \mu_k)^T = \sum_{k=0}^K (X^k - \mu_k)(X^k - \mu_k)^T$$

Covariance within class
(how much samples are dispersed in the class)

X^k is the matrix with columns the samples belonging to class k

$$S_B = \sum_{k=0}^K m_k (\mu_k - \mu)(\mu_k - \mu)^T$$

Covariance among classes
(how much classes are separated)

$$Z = W^T X$$

X is a matrix with size number_of_dimensions x number_of samples
W is a matrix number_of_dimensions x number_of_projections used to change coordinate system

$$S'_w = \sum_{k=0}^K (W^T X^k - W^T \mu_k)(W^T X^k - W^T \mu_k)^T = W^T \sum_{k=0}^K (X^k - \mu_k)(X^k - \mu_k)^T W = W^T S_w W$$


Covariance within class in the new coordinate system. The same principle can be used to write the covariance among classes (S_B) in the new coordinate system

$$J(W) = \frac{|W^T S_B W|}{|W^T S_W W|}$$

- The numerator is high if the classes are well separated (the determinant measures the "volume" of a covariance matrix)
- The denominator is low if the classes are compact
- A good transformation for separating the class is one that maximizes this function

αW

if we multiply W by a scalar constant, both the numerator and the denominator change in the same way

→ we can define W up to a normalization factor. So we can impose the denominator to be equal to 1 and maximize the numerator

By using the method of Lagrange multipliers $L(W, \lambda) = |W^T S_B W| - \lambda(|W^T S_W W| - 1)$

Let's consider the case of a 1D projection $L(W, \lambda) = W^T S_B W - \lambda(W^T S_W W - 1)$

$$\frac{\partial L}{\partial W} = 2S_B W - 2\lambda S_W W$$

$$S_B W = \lambda S_W W$$

setting the derivative to zero

$$S_W^{-1} S_B W = \lambda W$$

meaning that the derivative is zero when W is an eigenvector of $S_W^{-1} S_B$

$$J(W) = \frac{W^T S_B W}{W^T S_W W} = \frac{\lambda W^T S_W W}{W^T S_W W} = \lambda$$

$$S_B W = \lambda S_W W$$

So the highest eigenvalue is the one that maximize the function

- In case we want to project along different directions we can apply the same argument to the following eigenvectors
- The matrix S_B is a linear combination of rank-1 matrixes (products of one vector by its transpose is rank-1)
- Since the values μ_K and μ are linearly related, the maximum rank of S_B is $K-1 \rightarrow$ it is possible to project at maximum over a space with dimensions equal to the number of classes minus 1

MultiDimensional Scaling (MDS)

Dimensionality reduction technique that embeds data points into a lower-dimensional space while preserving the pairwise distances between points as closely as possible.

It needs as input the matrix of pairwise distance, so it can be used even when the actual coordinates of the samples are unknown.

If Euclidean distances are used, it is identical to PCA.

D^2

matrix of distances squared

$$d_{ij}^2 = |x^i - x^j|^2 = |x^i|^2 + |x^j|^2 - 2 x^i \cdot x^j$$

$$x^i \cdot x^j = -\frac{1}{2}(d_{ij}^2 - |x^i|^2 - |x^j|^2)$$

$$B = -\frac{1}{2}H(D^2)H$$

$$H = I - \frac{1}{m}\mathbf{1}\mathbf{1}^T$$

this operation removes the average from columns and rows, so it transforms the D^2 matrix into a matrix of inner products

$$AH = A \left(I - \frac{1}{n} \mathbf{1} \mathbf{1}^T \right) = A - \frac{1}{n} \begin{pmatrix} a_{11} & \dots & a_{1n} \\ \vdots & & \vdots \\ a_{n1} & \dots & a_{nn} \end{pmatrix} \begin{pmatrix} 1 & \dots & 1 \\ \vdots & \ddots & \vdots \\ 1 & \dots & 1 \end{pmatrix} = A - \frac{1}{n} \begin{pmatrix} \sum_j a_{1j} & \dots & \sum_j a_{1j} \\ \vdots & & \vdots \\ \sum_j a_{nj} & \dots & \sum_j a_{nj} \end{pmatrix}$$

The right product by H subtracts the row averages. In the same way it's possible to prove that the left product by H subtract the column averages.

$$B = X^T X = V \Lambda V^T$$

eigendecomposition

$$X = \Lambda^{1/2} V^T$$

In this way, we can reconstruct the original coordinates starting from the matrix of distances (with arbitrary origin of the reference system, due to the double centering procedure)

By selecting only the first p -dominant eigenvector, we project on the reduced space that better preserves the pair-wise distances

$$X' = \Lambda_p^{1/2} V_p^T$$

IsoMap

- For each data point connects it to its k nearest neighbours (or within a radius ϵ). This forms a graph where:
 - Nodes = data points
 - Edges = local distances
- Compute shortest paths between all pairs of points on the graph.
Possible algorithms: Dijkstra, Floyd–Warshall.
- Apply Multidimensional Scaling (MDS) to these distances

Locally Linear Embedding (LLE)

LLE assumes that high-dimensional data lies on a lower-dimensional manifold and that each data point can be approximated by a linear combination of its nearest neighbours

ALGORITHM

- For each point find the K nearest neighbours
- Approximate the point as a linear combination of its neighbours
- Find the projection Z that is best reconstructed by the weights defined in the previous step

- The local geometry of the data is preserved
- The global distances are not

$$x^i \approx \sum_k w_{i,k} x^k$$

$$f(W) = \sum_i \text{dist}(x^i - \sum_k w_{i,k} x^k)$$

minimize f with this constraint: $\sum_k w_{i,k} = 1$

$$g(Z) = \sum_i \text{dist}(z^i - \sum_k w_{i,k} z^k)$$

t-Distributed Stochastic Neighbour Embedding (t-SNE)

- AIM: project the data into a lower dimensional space where samples that are similar in the original space are close to each other in the reduced space, and samples that are dissimilar in the original space are distant in the reduced space
- Convert similarities into joint probabilities
- Minimize the Kullback-Leibler divergence between the joint probabilities

Kullback-Leibler divergence

$$D_{KL}(P|Q) = \sum_i P_i \log_2 \frac{P_i}{Q_i}$$

P and Q are discrete probability distributions

- D_{KL} measures the difference between the two distributions
- It's not a metric (e.g. it's not symmetric)
- If P_i is the real distribution and Q is a model approximation, DKL measure the lost information when Q is used to approximate P

Measure of similarity in the original space

$$P_{j|i} = \frac{e^{-\frac{d_{ij}^2}{2\sigma_i^2}}}{\sum_{k \neq i} e^{-\frac{d_{ik}^2}{2\sigma_i^2}}}$$

- Each samples is like a source of probability with a gaussian distribution
- The probability of j given i is the ratio between the probability that i generates j and the probability that i generated any other sample
- σ_i defines the width of the gaussian distribution
→ lower σ_i corresponds to a finer grid

$$p_{i,j} = \frac{P_{j|i} + P_{i|j}}{2N}$$

- Similarity between samples i and j

Definition of the variance

$$\textit{Perplexity}(P) = 2^{\textit{Entropy}(P)} = 2^{-\sum_i P_i \log_2 P_i} = \prod_i P_i^{-P_i}$$

Entropy: minimum number of bits
needed to encode the information

- Fix the perplexity
- For each point find the σ_i value that gives that perplexity
- In this way σ_i adapts to the local density of samples (lower in denser regions)

Perplexity: total information
encoded in the random variable

Measure of similarity in the low-dimensional space

$$q_{i,j} = \frac{(1 + d_{ij}^2)^{-1}}{\sum_k \sum_{l \neq k} (1 + d_{kl}^2)^{-1}}$$

- Similar to the measure of similarity in the original space but using the t-student distribution with 1 degree of freedom instead of the gaussian distribution
- Why ? Because the t-student distribution has a longer tail than the gaussian distribution, and this prevents an excessive crowding of the samples in the low-dimensional space

t-SNE (simplified) algorithm

- Find σ_i that fits the perplexity value (input parameter of the algorithm)
- Define random position for the samples in the low-dimensional space
- Use steepest descent to minimize D_{KL} (move the points in the direction opposite to the gradient of D_{KL} ; D_{KL} is not a convex function)

Uniform Manifold Approximation and Projection (UMAP)

- Maps a high-dimensional data set to a low-dimensional representation
 - The aim is to reveal clustering in the data
1. Build a graph for each point including the k-neighbors
 2. Merge the graphs into a single graph
 3. Create a low dimensional representation of the data (usually 2D)

1. Graphs of neighbors

- For each point (x) find the k nearest neighbours ($y_1, y_2, \dots, y_k$)
- Define a graph with links from x to y_i ($i = 1, \dots, k$)
- Define the weight of the link x - y_i so that:
 - the sum of the links is equal to $\log_2(k)$ for all the graphs
 - the weight of the link to the nearest point (y_1) is equal to 1
 - the weight of the links to the other points is proportional to the $\text{distance}(x, y_i) - \text{distance}(x, y_1)$

Intuition: the link x - y_i measures the probability that x and y_i belongs to the same cluster. By forcing the sum of weights to be constant, we are stretching the space to have uniform distribution of data.

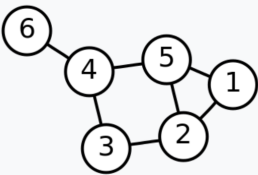
2. Merge the graph of neighbours

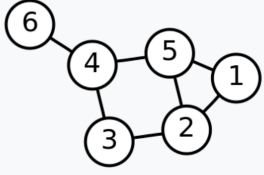
- Considering two random samples z_1 and z_2 , only 3 possibilities exist:
 - a) the pair z_1, z_2 does not appear in any graph
 - b) it appears in only one graph
 - c) it appears in 2 graphs (the ones respectively with centre z_1 and z_2)
- Case a) is trivial: no link between z_1 and z_2
- Case b) is also trivial: link between z_1 and z_2 with weights corresponding to the only value available
- Set weight is case c) as: $1 - (1-p_1)*(1-p_2)$, with p_1 and p_2 corresponding to the weights in the two graphs where the pair z_1 and z_2 appear

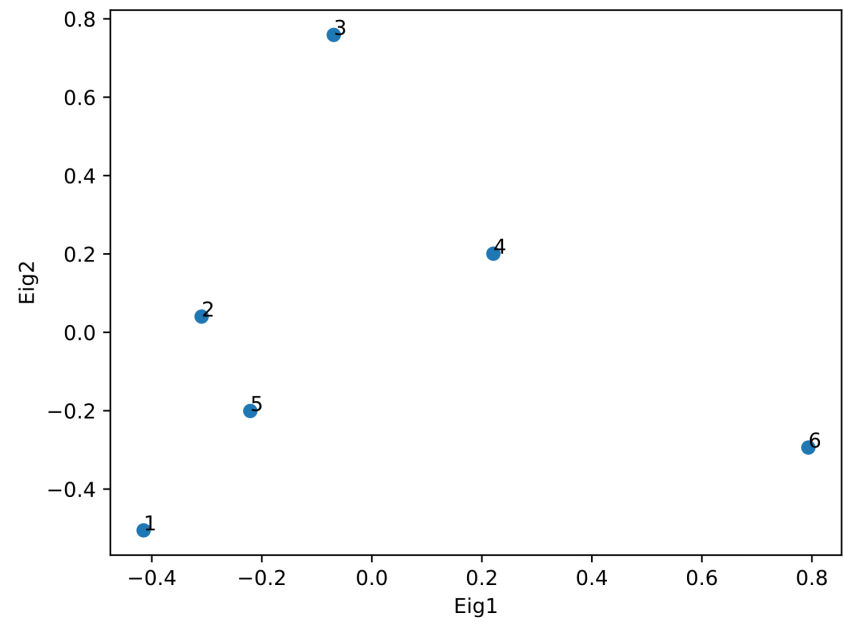
Intuition: $(1-p_1)$ can be interpreted as the probability that z_1 and z_2 are not linked in one of the graphs. The same for $(1-p_2)$ with respect to the other graph. So considering them as independent event $(1-p_1)*(1-p_2)$ is the probability that in both graphs the two samples are not linked. 1 minus this value estimates the probability that they are linked.

3a. Create an initial representation of the graph in a low dimensional (2D) space

- Define the Laplacian matrix of the graph (= degree matrix – adjacency matrix)
- Compute the first two non-trivial eigenvectors of the Laplacian matrix
- Take the N components of these eigenvectors as initial coordinates of the samples

Labelled graph	Degree matrix	Adjacency matrix	Laplacian matrix
	$\begin{pmatrix} 2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 3 & 0 & 0 \\ 0 & 0 & 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix}$	$\begin{pmatrix} 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{pmatrix}$	$\begin{pmatrix} 2 & -1 & 0 & 0 & -1 & 0 \\ -1 & 3 & -1 & 0 & -1 & 0 \\ 0 & -1 & 2 & -1 & 0 & 0 \\ 0 & 0 & -1 & 3 & -1 & -1 \\ -1 & -1 & 0 & -1 & 3 & 0 \\ 0 & 0 & 0 & -1 & 0 & 1 \end{pmatrix}$

Labelled graph	Degree matrix	Adjacency matrix	Laplacian matrix
	$\begin{pmatrix} 2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 3 & 0 & 0 \\ 0 & 0 & 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix}$	$\begin{pmatrix} 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{pmatrix}$	$\begin{pmatrix} 2 & -1 & 0 & 0 & -1 & 0 \\ -1 & 3 & -1 & 0 & -1 & 0 \\ 0 & -1 & 2 & -1 & 0 & 0 \\ 0 & 0 & -1 & 3 & -1 & -1 \\ -1 & -1 & 0 & -1 & 3 & 0 \\ 0 & 0 & 0 & -1 & 0 & 1 \end{pmatrix}$



3b. Improve the representation of the graph in the low dimensional space

- Use cross-entropy to estimate the difference between the graph and its low dimensional representation
- Minimize the cross-entropy

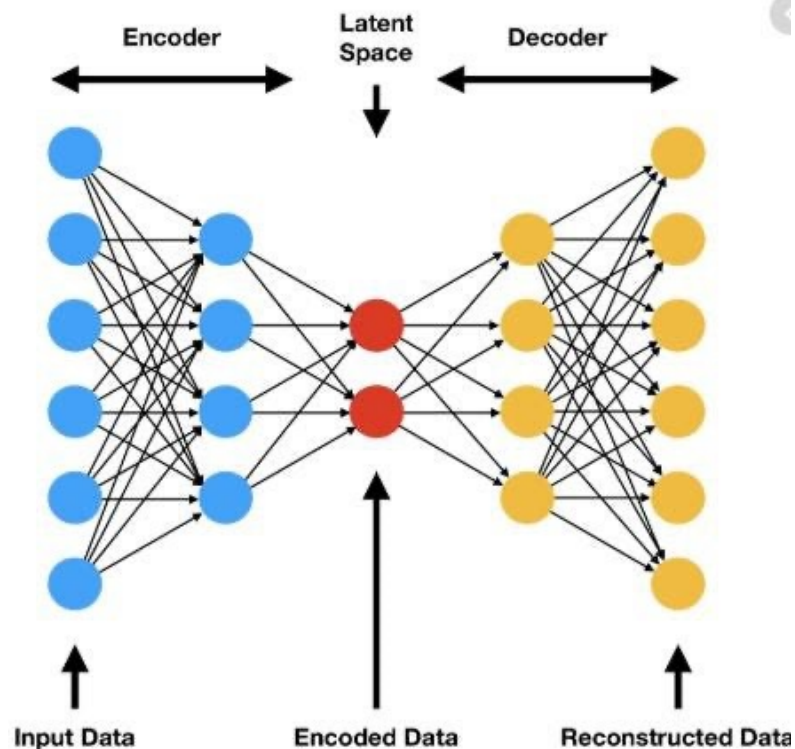
$$S = - \sum_{e \in E} p_e \log \frac{p_e}{q_e} - (1 - p_e) \log \frac{(1 - p_e)}{(1 - q_e)}$$

set of edges

probability
that the edge
exist in the
original graph

probability that the edge exist
in the low dimensional
representation

AutoEncoders for Dimensionality Reduction



- The ANN is trained to minimize the distance between input data and reconstructed data
- The latent space might be poorly regularized (e.g. sample that are close in the latent spaces might corresponds to input data highly different)
- Variational AutoEncoders are a possible solution to foster regularization of the latent space

Quantitative evaluation of nonlinear methods for population structure visualization and inference

Jordan Ubbens¹,^{1,*} Mitchell J. Feldmann²,^{2,†} Ian Stavness³,³ Andrew G. Sharpe¹

¹Global Institute for Food Security (GIFS), University of Saskatchewan, Saskatoon, SK S7N 0W9, Canada,

²Department of Plant Sciences, University of California, Davis, CA 95616, USA,

³Department of Computer Science, University of Saskatchewan, Saskatoon, SK S7N 0W9, Canada

*Corresponding author: Global Institute for Food Security (GIFS), University of Saskatchewan, Saskatoon, SK S7N 0W9, Canada. Email: jordan.ubbens@gifs.ca

[†]These authors contributed equally to this work.

